Lab 3: LU Factorization vs. Matrix Inversion

`CAS CS 132`: *Geometric Algorithms*

Due October 29, 2026 by 8:00PM

Outline. This lab you'll be benchmarking LU factorization and matrix inversion as implemented in NumPy and SciPy. You're required to submit a write-up for this lab as a PDF on Gradescope. The details of what you're required to submit are given in the last section called "Lab Write-up."

Introduction

In previous labs, we've set up the benchmarking code for you. This lab, you'll be setting it up yourselves.

Benchmarking basics. There are several ways to benchmark code in Python. We'll be using the simplest approach (which is also the approach that we took for Labs 1 and 2). The Python module `time` has a function called `time` which can be used to get the current time. We can use this to determine how much time has been elapsed since a process was started:

```
start = time.time()
process() # thing being timed
total = time.time() - start
```

It's important `start` is determined *just* before the process you want to time. You don't want to include any proprocessing in the time measured.

Randomness. As a reminder, when we benchmark a function, we usually test it on random inputs. We'll use a `numpy.random.generator` to generate these inputs. To create a random number generator, we can use the constructor given in NumPy:

```
rng = np.random.default_rng()
```

We can then use the `uniform` method to construct random matrices that we'll use for benchmarking:

```
# m by n matrix with numbers in [low, high)
a = rng.uniform(low, high, (m, n))

# n elment vector with numbers in [low, high)
b = rng.uniform(low, high, n)
```

Solving matrix equations. By now we've seen three ways to solve matrix equations of the form $A\mathbf{x} = \mathbf{b}$. Each of these ways corresponds to a function in Numpy/Scipy:

- (`numpy.linalg.solve`) use Gaussian elimination on the matrix $[A\ \mathbf{b}]$; this was the focus of Lab 2;
- (`scipy.linalg.lu_factor` and `scipy.linalg.lu_solve`) compute the LU Factorization of A and use this to solve $A\mathbf{x} = \mathbf{b}$;
- (`numpy.linalg.inv`) determine the inverse A^{-1} and compute $\mathbf{x} = A^{-1}\mathbf{b}$.

It's conventional wisdom that you should always use LU factorization, particularly if you need to compute the same matrix equation with different right hand sides. This is especially true if the matrix is well-structured (one of the "arts" of solving matrix equations with computers is that different approaches may be more or less efficient depending on the shape and structure of A). We'll be looking into this phenomenon by benchmarking the three approaches above.

Lab Write-Up

This section contains the tasks you're required to complete for this lab. Some questions require you to write code, others require you to reason about code. If a question is labeled with a "(*)", it's the kind of question that could appear on a quiz or on an exam. The parts in **bold** are what must be included in your write-up.

1. (*) Implement the three processes that we want to benchmark. Each one should be of the form

   ```
   process(a, bs)
   ```

   where `a` is matrix and `bs` is a list of vectors. The output should be a list of solutions, one for each right-hand side in `bs`.

   - `solve(a, bs)` which uses `numpy.linalg.solve` to solve each matrix equation;
   - `lu_solve(a, bs)` which uses the function `scipy.linalg.lu_factor` to LU factor `a`, and then uses `scipy.linalg.lu_solve` to solve each matrix equation; Important: this should only factor once;
   - `inv_solve(a, bs)` which uses `numpy.linalg.inv` to invert `a`, and then uses uses matrix-vector multiplication (`@`) to solve each matrix equation; Important: this should only invert once.

   In each case you should iterate through the vectors in `bs` and perform the respective solving method.

   Include your implementations in your write-up.

2. Implement the function `benchmark_random` in the starter code. Look at the docstring to see how this function should work.

   There is nothing to include in your write-up for this part.

3. Read through the function `banded_matrix` in the starter code and try to understand what it is doing.

   Include in the value `banded_matrix(3)` in your write-up.

4. Implement the function `benchmark_banded` in the starter code. Look at the docstring to see how this function should work.

   There is nothing to include in your write-up for this part.

5. Using the function `benchmark_random`, create a plot which compares the running time of `solve` and `lu_solve` for between 80 and 100 experiments (depending on what your machine can handle) with step size 10, where each experiment uses the given process to solve 100 equations. Your plot should have a title, labeled axes, and a legend.

   Include the graph in your lab write-up. Explain in 1-2 sentences what this graph indicates.

6. Using the function `benchmark_random`, create a plot which compares the running time of `inv_solve` and `lu_solve` for between 200 and 300 experiments (depending on what your machine can handle) with step size 10, where each experiment uses the given process to solve 100 equations. Your plot should have a title, labeled axes, and a legend.

   Include the graph in your lab write-up. Explain in 1-2 sentences what this graph indicates.

7. Using the function `benchmark_banded`, create a plot which compares the running time of `inv_solve` and `lu_solve` for between 8 and 12 experiments (depending on what your machine can handle) with step size 1000, where each experiment uses the given process to solve 100 equations. Your plot should have a title, labeled axes, and a legend.

   Include the graph in your lab write-up. Explain in 1-2 sentences what this graph indicates.